

A Bounded Binary64 Exponential

LeanExe library PoC

September 2026

1 Task and numerical domain

`expTaylor6` approximates e^x on $[-1, 0]$ using the degree-six Taylor polynomial

$$p_6(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + \frac{x^6}{720}.$$

The exponential power series appears in DLMF equation 4.2.19 [1]. On this interval its alternating tail has magnitude at most $1/5040$ in exact arithmetic. The target bound for the binary64 computation is $1/4000$.

`expTaylor6` accepts and returns binary64 bit patterns in `UInt64` words. The result is optional: a finite input in $[-1, 0]$ returns a value, and other inputs return `none`. Both signed zeros return the exact bits of one. Negative subnormals are admitted. Infinities and NaNs are rejected by the bit-range check. LeanExe compiles the Lean function and its callers to WebAssembly (WASM). The examples run that WASM in Wasmtime.

2 Verification status

Lean checks the source types. Execution tests pass for selected inputs: WASM and native Lean agree bit for bit, domain checks give the expected status, and accepted results lie within $1/4000$ of JavaScript's `Math.exp`.

A separate Lean theorem, `Project.ExpSmall.Spec.expSmall_real_error`, proves termination and a finite positive result within $1/4000$ of the real exponential for every finite binary64 input in $[-1, 0]$ [2]. Its subject is the WASM model `Project.ExpSmall.module`. Formal verification of this library function and its compiled clients, including their connection to that theorem, remains deferred.

3 Implementation and coefficient choices

```
import LeanExe.Float64

namespace LeanExe.Lib.Transcendental

def expTaylor6Polynomial (x : UInt64) : UInt64 :=
  let a := Float64.addBits (Float64.mulBits 0x3F56C16C16C16C17 x) 0x3F81111111111111
  let b := Float64.addBits (Float64.mulBits a x) 0x3FA5555555555555
  let c := Float64.addBits (Float64.mulBits b x) 0x3FC5555555555555
  let d := Float64.addBits (Float64.mulBits c x) 0x3FE0000000000000
```

```

let e := Float64.addBits (Float64.mulBits d x) 0x3FF0000000000000
Float64.addBits (Float64.mulBits e x) 0x3FF0000000000000

def expTaylor6 (bits : UInt64) : Option UInt64 :=
  if bits == 0 || (bits >= 0x8000000000000000 && bits <= 0xBFF0000000000000) then
    some (expTaylor6Polynomial bits)
  else none

end LeanExe.Lib.Transcendental

```

The listing comes from `Basic.lean`. Each multiplication and addition rounds separately. The hexadecimal literals encode the rounded coefficients. `expTaylor6Polynomial` exposes the unguarded evaluator. `expTaylor6` applies the domain guard before calling it. Evaluation uses six multiplications and six additions.

The coefficient words, evaluation order, and domain follow `Project.ExpSmall.Model`. This component expresses the arithmetic through `LeanExe.Float64` primitives.

4 A decay client

The decay client approximates the remaining fraction e^{-t} for dimensionless time $t \in [0, 1]$. It checks t , negates its binary64 sign bit, and calls the exponential component. The commands run from the repository root after following the setup guide.

```

tools/seminum exp-taylor6 -0.5
0.6065321180555556
tools/seminum decay 0.5
0.6065321180555556
tools/seminum exp-taylor6-bits bfe0000000000000
3fe368b60b60b60c

```

The host converts decimal inputs to binary64 and formats returned values in decimal. The hexadecimal command preserves input and output bits. Out-of-domain inputs cause exit status 2 and a diagnostic on standard error.

5 Tests

From the repository root, `node test/seminum.js` runs the comparisons described above at 33 equally spaced points in $[-1, 0]$, both signed zeros, subnormals, adjacent boundary words, infinities, and NaNs. Accepted outputs are finite and positive on these cases. The largest sampled absolute difference from `Math.exp` is about $1.7611438411335723 \times 10^{-4}$ at $x = -1$, within the target $1/4000$. The decay client has corresponding comparisons on $[0, 1]$. CLI tests check decimal and hexadecimal output and rejected input. The cases are defined in `test/seminum.js` and the native reference driver `test/SeminumNative.lean`.

References

- [1] NIST Digital Library of Mathematical Functions, equation 4.2.19. <https://dlmf.nist.gov/4.2.E19>. The power series supplies the coefficients before rounding and truncation.
- [2] LeanExe, *Small-exponential execution theorem*. The numerical proof bounds polynomial approximation, coefficient rounding, and arithmetic rounding for `Project.ExpSmall.module`.